

Python Round 1 Interview Questions & Detailed Answers

Experience – 0-5 Year

1. What are the most important Python data structures used in ETL pipelines?

Answer:

Common and essential structures include:

- **List:** Sequential data, buffering rows
- **Tuple:** Immutable values like config settings
- **Dict:** JSON-like records, row-wise data
- **Set:** Fast deduplication
- **Deque:** Efficient for streaming ingest, append/pop operations

Real Scenario:

While processing S3 access logs, each event is parsed into a dict, lists are used to batch records, and sets help eliminate duplicates.

2. How do you handle large datasets (10GB+) in Python without memory errors?

Large files cannot fit into memory. Use **streaming** and **chunking**.

Solutions:

Chunking using Pandas

```
for chunk in pd.read_csv("big.csv", chunksize=50000):  
    process(chunk)
```

Use Generators

```
def read_large_file(path):  
    with open(path) as f:  
        for line in f:  
            yield line
```

Use efficient libraries:

- Dask
- Polars
- PyArrow

Push to distributed systems: Spark on Databricks/EMR

Real Scenario:

A 15GB file causing memory failure in Pandas was handled successfully using chunk reads.

3. What are Python generators and why are they used in ETL pipelines?

Answer:

Generators produce values one at a time without loading everything into memory.

Example:

```
def stream_rows(file):
    for row in file:
        yield row
```

Use Cases:

- Streaming logs
- Kafka consumption
- Large file ETL
- Memory-efficient transformations

4. Explain decorators with a real-time ETL example

Decorators modify the behavior of a function without changing its internal code.

Example: Logging ETL steps

```
def log_step(func):
    def wrapper(*args, **kwargs):
        print(f"Running step: {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_step
def extract_data():
    return "data"
```

Uses:

- Logging
- Authentication
- Retry logic
- Performance monitoring

5. What is the difference between multiprocessing and multithreading?

Multithreading:

- Best for I/O-bound tasks
- Affected by the Global Interpreter Lock (GIL)
- Used for API calls, file read/write

Multiprocessing:

- Best for CPU-bound tasks
- Bypasses GIL
- Each process has its own memory and interpreter

Example:

```
from multiprocessing import Pool

def transform(row):
    return row * 10

with Pool(4) as p:
    result = p.map(transform, data)
```

6. How do you optimize a slow Python ETL job?

Common Techniques:

1. Eliminate Python loops → use Pandas/NumPy vectorization
2. Use caching (Redis, local cache)
3. Use bulk inserts instead of row-by-row writes
4. Reuse database connections
5. Reduce serialization overhead
6. Switch CSV → Parquet for faster processing
7. Profile code using:

```
python -m cProfile etl.py
```

Real Example:

An ETL reduced from 2 hours to 20 minutes by batching SQL writes and removing loops.

7. How do you validate data in Python before loading it?

Use:

- Conditional checks
- Schema validation libraries
- Pydantic
- Great Expectations

Example using Pydantic:

```
from pydantic import BaseModel, ValidationError

class User(BaseModel):
```

```

        id: int
        name: str
        email: str

    try:
        user = User(**row)
    except ValidationError as e:
        log_error(e)

```

8. How do you connect Python to a database and perform bulk inserts?

PostgreSQL Fast Load:

```

import psycopg2

conn = psycopg2.connect(...)
cur = conn.cursor()

cur.execute("COPY table_name FROM STDIN WITH CSV", file)
conn.commit()

```

Batch Inserts:

```

cur.executemany(
    "INSERT INTO sales VALUES (%s,%s,%s)",
    batch_rows
)

```

Best Practices:

- Use connection pooling
- Keep batch size optimal (5k–10k rows)
- Use prepared statements

9. Explain exception handling in ETL pipelines with examples

Example:

```

try:
    result = process(row)
except ValueError as e:
    log("Bad row", e)
except ConnectionError:
    retry()
finally:
    clean_temp_files()

```

Use Cases:

- API failure
- Missing columns

- Corrupted rows
- Connection timeout

10. How do you handle schema drift in incoming JSON data?

Solutions:

- Maintain schema registry
- Add missing keys
- Alert for unexpected data
- Normalize dynamic fields

Example:

```
for record in data:
    for col in master_schema:
        record.setdefault(col, None)
```

11. What is the difference between shallow copy and deep copy?

Shallow Copy:

- Copies object references
- Changes in nested objects reflect across copies

Deep Copy:

- Copies actual nested objects
- Independent memory allocation

Example:

```
import copy
a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
```

12. How do you process multi-GB JSON files?

Methods:

- Use ijson (streaming JSON parser)
- Use JSONLines format
- Use gzip + stream read

Example (ijson):

```
import ijson
```



```
for item in ijson.items(open("big.json"), "records.item"):
    process(item)
```

13. How do you schedule Python ETL jobs?

Options:

- Cron
- Airflow
- Prefect
- AWS Lambda with EventBridge
- Azure Functions

Cron Example:

```
0 * * * * python etl.py
```

14. Explain context managers and their use in pipelines

Example:

```
with open("file.txt") as f:
    process(f)
```

Benefits:

- Automatic cleanup
- No file handle leaks
- Used for DB connections, file operations, S3 streams

15. Explain list vs generator performance in ETL

List:

- Loads all data into memory
- Fast random access
- Not suitable for large files

Generator:

- Lazy loading
- Very low memory usage
- Ideal for large data streaming